SISTEMAS DISTRIBUIDOS

Procesamiento distribuido Parte 2

CLASE 22 | REPORTE DE PRÁCTICA

Integrantes

GONZALEZ MARTINEZ ALBERTO EZEQUIEL
MENDOZA CASTAÑEDA JOSE RICARDO

Profesor y grupo

UKRANIO CORONILLA CONTRERAS
7CM3

Fecha de entrega

26 de octubre de 2026

Cliente asíncrono, coordinación de dos workers, endpoint /dispatcher e interfaz web mediante NGINX.

Objetivo y funcionamiento

Implementar un sistema de procesamiento distribuido que reciba un hash SHA-256 y busque la palabra de cinco letras minúsculas que lo produce. El coordinador divide la búsqueda en 26 tareas y las distribuye dinámicamente entre dos servidores trabajadores.

Componente	Función
Application	Genera las 26 tareas y ejecuta el cliente de consola.
Aggregator	Asigna la siguiente tarea al worker disponible y reúne resultados.
networking.WebClient	Envía peticiones POST asíncronas a /task.
WebServer :8080	Recibe el hash en /dispatcher y entrega el resultado y su tiempo.
Workers :8081 y :8082	Prueban las combinaciones asignadas a una primera letra.
NGINX :8090	Sirve la interfaz y reenvía /dispatcher al coordinador.

División del espacio de búsqueda

Cada tarea tiene el formato **hashObjetivo,letra**. Al fijar la primera letra, cada worker explora hasta 26^4 = 456,976 candidatos. Las 26 tareas cubren las 26^5 = 11,881,376 combinaciones de cinco letras a-z.

Parada y tiempo total

Cuando un worker encuentra una palabra válida, se detienen los nuevos envíos. Las tareas ya enviadas terminan y sus resultados también se incluyen. Por ello pueden aparecer la contraseña y un resultado NULL de otro worker. El tiempo total incluye la espera de las solicitudes en curso.

Organización del reporte

Se presenta el código actual de los ejercicios 1, 2 y 3, sus evidencias y la configuración de NGINX utilizada en WSL. Las pruebas iniciales conservadas del documento original se distinguen de la ejecución final con abaco. Los fragmentos del worker se incluyen como apoyo del procesamiento.

Ejercicio 1 | Application

Genera las tareas mediante Aggregator.createTasks y mide el proceso con System.nanoTime. Recibe un hash como primer argumento; sin argumento utiliza el hash de abaco. Puede recibir URLs de workers adicionales después del hash.

source/Application.java

```
1  import java.util.*;
2
3  public class Application {
4      public static void main(String[] args) throws Exception {
5          String hash = args.length > 0 ? args[0] : FuerzaBrutaHash.sha256("abaco");
6          List<String> workers = args.length > 1 ?
   Arrays.asList(Arrays.copyOfRange(args, 1, args.length))
7                  : List.of("http://localhost:8081/task", "http://localhost:8082/task");
8          List<String> tasks = Aggregator.createTasks(hash);
9          long start = System.nanoTime();
10         List<Aggregator.Result> results = new Aggregator().sendTasksToWorkers(workers,
    tasks);
11         System.out.print(Aggregator.report(results, (System.nanoTime() - start) /
   1e9));
12     }
13 }
```

Código fuente actual reproducido como imagen. Las líneas largas se ajustan visualmente; el archivo original no se modifica.

Ejercicio 1 | WebClient

Cliente HTTP asíncrono del paquete networking. Envía el contenido hash,letra mediante POST, aplica tiempos límite y comprueba el estado HTTP antes de devolver el cuerpo de la respuesta.

source/WebClient.java

```
1   package networking;
2
3   import java.net.URI;
4   import java.net.http.*;
5   import java.time.Duration;
6   import java.util.concurrent.CompletableFuture;
7
8   public class WebClient {
9       private final HttpClient client = HttpClient.newBuilder()
10              .connectTimeout(Duration.ofSeconds(5)).build();
11
12      public CompletableFuture<String> sendTask(String url, String task) {
13          HttpRequest request = HttpRequest.newBuilder(URI.create(url))
14                  .timeout(Duration.ofMinutes(3))
15                  .header("Content-Type", "text/plain; charset=UTF-8")
16                  .POST(HttpRequest.BodyPublishers.ofString(task)).build();
17          return client.sendAsync(request, HttpResponse.BodyHandlers.ofString())
18                  .thenApply(response -> {
19                      if (response.statusCode() != 200)
20                          throw new IllegalStateException("Worker " + url + ": HTTP " +
    response.statusCode());
21                      return response.body().trim();
22                  });
23      }
24  }
```

Código fuente actual reproducido como imagen. Las líneas largas se ajustan visualmente; el archivo original no se modifica.

Ejercicio 1 | Aggregator (1/3)

Versión final: cada consulta usa su propio estado sincronizado. Se generan 26 tareas, se mantiene una cadena por worker y se validan las palabras recibidas. allOf(...).join() espera las tareas ya enviadas antes de construir el reporte.

source/Aggregator.java

```
1  import networking.WebClient;
2  import java.util.*;
3  import java.util.concurrent.*;
4
5  /** Una cadena asincrona por worker; el estado pertenece a una sola consulta. */
6  public class Aggregator {
7      private final WebClient client = new WebClient();
8
9      public static List<String> createTasks(String hash) {
10         if (hash == null || !hash.matches("[0-9a-fA-F]{64}"))
11             throw new IllegalArgumentException("El hash debe tener 64 caracteres
   hexadecimales");
12         List<String> tasks = new ArrayList<>();
13         for (char letter = 'a'; letter <= 'z'; letter++) tasks.add(hash + "," +
   letter);
14         return tasks;
15     }
16
17     public static class Result {
18         public final String task, worker, value;
19         public final int index;
20         Result(int index, String task, String worker, String value) {
21             this.index = index; this.task = task; this.worker = worker; this.value =
   value;
22         }
23     }
24
25     public List<Result> sendTasksToWorkers(List<String> workers, List<String> tasks) {
26         if (workers.isEmpty()) throw new IllegalArgumentException("Se requiere al
   menos un worker");
27         State state = new State(tasks);
28         CompletableFuture<?>[] chains = new CompletableFuture<?>[workers.size()];
29         for (int i = 0; i < workers.size(); i++) chains[i] = next(workers.get(i),
   state);
30         // Se dejan terminar las tareas ya enviadas para devolver sus resultados
   tambien.
31         CompletableFuture.allOf(chains).join();
32         state.results.sort(Comparator.comparingInt(r -> r.index));
33         return state.results;
34     }
35
36     private static class State {
37         final List<String> tasks;
```

Código fuente actual reproducido como imagen. Las líneas largas se ajustan visualmente; el archivo original no se modifica.

Ejercicio 1 | Aggregator (2/3)

Continuación del mismo archivo. La numeración izquierda corresponde a las líneas del código fuente.

source/Aggregator.java

```
38          final List<Result> results = new ArrayList<>();
39          int index;
40          boolean stopped;
41          State(List<String> tasks) { this.tasks = List.copyOf(tasks); }
42      }
43
44      private CompletableFuture<Void> next(String worker, State state) {
45          synchronized (state) {
46              if (state.stopped || state.index >= state.tasks.size())
47                  return CompletableFuture.completedFuture(null);
48              int index = state.index++;
49              String task = state.tasks.get(index);
50              System.out.printf("Tarea %d (%s) -> %s%n", index + 1, task, worker);
51              return client.sendTask(worker, task).handle((value, error) -> {
52                  synchronized (state) {
53                      if (error != null) {
54                          state.stopped = true;
55                          throw new CompletionException(error);
56                      }
57                      if (!value.equals("NULL")) {
58                          String hash = task.substring(0, 64);
59                          try {
60                              if (!value.matches("[a-z]{5}") || value.charAt(0) !=
    task.charAt(65)
61                                      ||
    !FuerzaBrutaHash.sha256(value).equalsIgnoreCase(hash))
62                                  throw new IllegalStateException("Respuesta invalida
    del worker " + worker);
63                          } catch (Exception ex) {
64                              state.stopped = true;
65                              throw new CompletionException(ex);
66                          }
67                          state.stopped = true;
68                      }
69                      state.results.add(new Result(index, task, worker, value));
70                      System.out.printf("Tarea %d terminada: %s%n", index + 1, value);
71                  }
72                  return value;
73              }).thenCompose(ignored -> next(worker, state));
74          }
75      }
76
77      public static String report(List<Result> results, double seconds) {
```

Código fuente actual reproducido como imagen. Las líneas largas se ajustan visualmente; el archivo original no se modifica.

Ejercicio 1 | Aggregator (3/3)

Continuación del mismo archivo. La numeración izquierda corresponde a las líneas del código fuente.

source/Aggregator.java

```
78          StringBuilder out = new StringBuilder();
79          String password = "NULL";
80          for (Result r : results) {
81              out.append(String.format("Tarea %d (%s) [%s]: %s%n", r.index + 1, r.task,
    r.worker, r.value));
82              if (!r.value.equals("NULL")) password = r.value;
83          }
84          out.append("Contrasena: ").append(password).append('\n');
85          out.append("Tareas realizadas: ").append(results.size()).append(" / 26\n");
86          out.append(String.format(Locale.ROOT, "Tiempo total: %.6f segundos%n",
    seconds));
87          return out.toString();
88      }
89  }
```

Código fuente actual reproducido como imagen. Las líneas largas se ajustan visualmente; el archivo original no se modifica.

Ejercicio 1 | Evidencia inicial

La captura original documenta una búsqueda de **gatos** con el cliente asíncrono: se encontró la palabra y se detuvo el envío de nuevas tareas. La medición visible es **0.153 s**. Corresponde a la versión inicial, cuya salida y argumentos eran distintos a los de la versión final documentada en las páginas anteriores.

```
→  classes java -cp Application.jar Application gatos
Hash objetivo: 7051fd3b01733aeda1293523ef951144c5af864b529774853205b8fb1670e877
Tarea 1 (letra a) asignada al servidor 8081
Tarea 2 (letra b) asignada al servidor 8082
El servidor 8082 terminó la tarea 2 (letra b): NULL
El servidor 8081 terminó la tarea 1 (letra a): NULL
Tarea 3 (letra c) asignada al servidor 8082
Tarea 4 (letra d) asignada al servidor 8081
El servidor 8082 terminó la tarea 3 (letra c): NULL
Tarea 5 (letra e) asignada al servidor 8082
El servidor 8081 terminó la tarea 4 (letra d): NULL
Tarea 6 (letra f) asignada al servidor 8081
El servidor 8082 terminó la tarea 5 (letra e): NULL
Tarea 7 (letra g) asignada al servidor 8082
El servidor 8082 terminó la tarea 7 (letra g): gatos
El servidor 8082 encontró la contraseña: gatos. Ya no se envían más tareas.

Tareas realizadas:
Tarea 1 (letra a): NULL
Tarea 2 (letra b): NULL
Tarea 3 (letra c): NULL
Tarea 4 (letra d): NULL
Tarea 5 (letra e): NULL
Tarea 7 (letra g): gatos

Contraseña encontrada: gatos
Tiempo total de procesamiento: 0.153 s
```

En la versión actual, para repetir una búsqueda de gatos se debe proporcionar su hash SHA-256 como argumento de Application. La ejecución final a través de /dispatcher y NGINX se muestra en el ejercicio 3.

Ejercicio 2 | WebServer (1/2)

Servidor HTTP con /dispatcher. Recibe el hash por POST como texto plano o por GET en la consulta. Valida las entradas, crea un agregador por solicitud y devuelve las tareas, la contraseña, el número de tareas y el tiempo total. Un fallo de worker produce HTTP 502.

source/WebServer.java

```
1  import com.sun.net.httpserver.*;
2  import java.net.*;
3  import java.nio.charset.StandardCharsets;
4  import java.util.*;
5  import java.util.concurrent.*;
6  import java.io.IOException;
7
8  public class WebServer {
9      private static void reply(HttpExchange e, int status, String text) throws
   IOException {
10         byte[] bytes = text.getBytes(StandardCharsets.UTF_8);
11         e.getResponseHeaders().set("Content-Type", "text/plain; charset=UTF-8");
12         e.sendResponseHeaders(status, bytes.length);
13         e.getResponseBody().write(bytes);
14     }
15
16     public static void main(String[] args) throws Exception {
17         int port = args.length > 0 ? Integer.parseInt(args[0]) : 8080;
18         List<String> workers = args.length > 1 ?
   Arrays.asList(Arrays.copyOfRange(args, 1, args.length))
19                 : List.of("http://localhost:8081/task", "http://localhost:8082/task");
20         HttpServer server = HttpServer.create(new InetSocketAddress("0.0.0.0", port),
   0);
21         server.createContext("/dispatcher", e -> {
22             try {
23                 if (!e.getRequestURI().getPath().equals("/dispatcher")) {
24                     reply(e, 404, "Ruta inexistente"); return;
25                 }
26                 String input;
27                 if (e.getRequestMethod().equals("POST")) {
28                     byte[] bytes = e.getRequestBody().readNBytes(4097);
29                     if (bytes.length > 4096) { reply(e, 413, "Cuerpo demasiado
   grande"); return; }
30                     input = new String(bytes, StandardCharsets.UTF_8).trim();
31                 } else if (e.getRequestMethod().equals("GET")) {
32                     input = e.getRequestURI().getRawQuery();
33                     if (input == null) input = "";
34                     input = URLDecoder.decode(input, StandardCharsets.UTF_8);
35                     if (input.startsWith("hash=")) input = input.substring(5);
36                 } else {
37                     e.getResponseHeaders().set("Allow", "GET, POST");
38                     reply(e, 405, "Use GET o POST"); return;
39                 }
```

Código fuente actual reproducido como imagen. Las líneas largas se ajustan visualmente; el archivo original no se modifica.

Ejercicio 2 | WebServer (2/2)

Continuación del mismo archivo. La numeración izquierda corresponde a las líneas del código fuente.

source/WebServer.java

```
40                 List<String> tasks = Aggregator.createTasks(input);
41                 System.out.println("Nueva consulta: " + input);
42                 long start = System.nanoTime();
43                 List<Aggregator.Result> results = new
   Aggregator().sendTasksToWorkers(workers, tasks);
44                 double seconds = (System.nanoTime() - start) / 1e9;
45                 e.getResponseHeaders().set("X-debug-info", String.format(Locale.ROOT,
   "seconds=%.6f", seconds));
46                 String report = Aggregator.report(results, seconds);
47                 System.out.print(report);
48                 reply(e, 200, report);
49             } catch (IllegalArgumentException ex) {
50                 reply(e, 400, ex.getMessage());
51             } catch (Exception ex) {
52                 ex.printStackTrace();
53                 reply(e, 502, "No fue posible completar la consulta con los workers");
54             } finally { e.close(); }
55         });
56         server.createContext("/", e -> {
57             try { reply(e, 404, "Use /dispatcher"); } finally { e.close(); }
58         });
59         server.setExecutor(Executors.newFixedThreadPool(8));
60         server.start();
61         System.out.println("Dispatcher en http://localhost:" + port + "/dispatcher");
62         System.out.println("Workers: " + workers);
63     }
64 }
```

Código fuente actual reproducido como imagen. Las líneas largas se ajustan visualmente; el archivo original no se modifica.

Ejercicio 2 | Evidencia inicial

Las capturas originales muestran el despachador y una consulta POST con curl. La búsqueda de **gatos** devolvió HTTP 200 y **0.226 s**. Son evidencias de la versión inicial del endpoint; el formato final de la respuesta se observa en el ejercicio 3.

```
→  classes java -cp WebServer.jar WebServer 8080
Servidor despachador escuchando en el puerto 8080
Endpoint: POST /dispatcher (cuerpo = hashObjetivo) o GET /dispatcher?<hashObjetivo>

Petición recibida en /dispatcher, hash objetivo: 7051fd3b01733aeda1293523ef951144c5af864b529774853205b8fb1670e877
Tarea 1 (letra a) asignada al servidor 8081
Tarea 2 (letra b) asignada al servidor 8082
El servidor 8081 terminó la tarea 1 (letra a): NULL
Tarea 3 (letra c) asignada al servidor 8081
El servidor 8082 terminó la tarea 2 (letra b): NULL
Tarea 4 (letra d) asignada al servidor 8082
El servidor 8081 terminó la tarea 3 (letra c): NULL
Tarea 5 (letra e) asignada al servidor 8081
El servidor 8082 terminó la tarea 4 (letra d): NULL
Tarea 6 (letra f) asignada al servidor 8082
El servidor 8082 terminó la tarea 6 (letra f): NULL
Tarea 7 (letra g) asignada al servidor 8082
El servidor 8082 terminó la tarea 7 (letra g): gatos
El servidor 8082 encontró la contraseña: gatos. Ya no se envían más tareas.
Hash objetivo: 7051fd3b01733aeda1293523ef951144c5af864b529774853205b8fb1670e877

Tareas realizadas:
Tarea 1 (letra a): NULL
Tarea 2 (letra b): NULL
Tarea 3 (letra c): NULL
Tarea 4 (letra d): NULL
Tarea 6 (letra f): NULL
Tarea 7 (letra g): gatos

Contraseña encontrada: gatos
Tiempo total de procesamiento: 0.226 s
```

```
→  Sistemas Distribuidos curl -i -X POST --data "$HASH" http://localhost:8080/dispatcher
HTTP/1.1 200 OK
Date: Fri, 23 Oct 2026 17:36:46 GMT
Content-type: text/plain; charset=utf-8
X-debug-info: seconds=0.226
Content-length: 315

Hash objetivo: 7051fd3b01733aeda1293523ef951144c5af864b529774853205b8fb1670e877

Tareas realizadas:
Tarea 1 (letra a): NULL
Tarea 2 (letra b): NULL
Tarea 3 (letra c): NULL
Tarea 4 (letra d): NULL
Tarea 6 (letra f): NULL
Tarea 7 (letra g): gatos

Contraseña encontrada: gatos
Tiempo total de procesamiento: 0.226 s

→  Sistemas Distribuidos
```

Ejercicio 3 | Interfaz web

Fragmentos funcionales del archivo actual: formulario, validación del hash, ejemplo de abaco y envío POST a /dispatcher. Se omiten únicamente los estilos visuales y las etiquetas meta iniciales; la numeración conserva las líneas del archivo.

web/index.html

```
9   <main>
10  <h1>Búsqueda distribuida SHA-256</h1>
11  <p>Busca una palabra de cinco letras minúsculas (a–z) mediante dos servidores
    trabajadores.</p>
12  <form id="form">
13  <label for="hash">Hash SHA-256 de la palabra</label>
14  <input id="hash" name="hash" required pattern="[0-9a-fA-F]{64}" minlength="64"
    maxlength="64" placeholder="64 caracteres hexadecimales" autocomplete="off">
15  <button id="search">Buscar contraseña</button>
16  <button type="button" id="example">Cargar ejemplo: abaco</button>
17  </form>
18  <small>El tiempo informado por el servidor incluye el reparto y la espera de las
    tareas ya enviadas.</small>
19  <pre id="result" role="status" aria-live="polite">Esperando una consulta.</pre>
20  </main>
21  <script>
22  const form=document.querySelector('#form'), hash=document.querySelector('#hash'),
    result=document.querySelector('#result'), button=document.querySelector('#search');
23  document.querySelector('#example').onclick=async()=>{
24   const digest=await crypto.subtle.digest('SHA-256',new TextEncoder().encode('abaco'));
25   hash.value=Array.from(new
    Uint8Array(digest),b=>b.toString(16).padStart(2,'0')).join('');
26  };
27  form.onsubmit=async event=>{
28   event.preventDefault(); button.disabled=true; result.textContent='Procesando la
    búsqueda en los workers…';
29   try{
30    const response=await
    fetch('/dispatcher',{method:'POST',headers:{'Content-Type':'text/plain;
    charset=UTF-8'},body:hash.value.trim()});
31    const text=await response.text();
32    result.textContent=response.ok?text:`Error HTTP ${response.status}: ${text}`;
33   }catch(error){result.textContent='No se pudo contactar al servidor: '+error.message;}
34   finally{button.disabled=false;}
35  };
36  </script>
```

Código fuente actual reproducido como imagen. Las líneas largas se ajustan visualmente; el archivo original no se modifica.

Ejercicio 3 | NGINX en WSL

Configuración utilizada en la ejecución: NGINX escucha en 8090, sirve la carpeta web y reenvía /dispatcher a 127.0.0.1:8080. La ruta absoluta corresponde a la computadora de la práctica y debe adaptarse al ejecutar en otra ubicación.

nginx/nginx-wsl.conf

```
worker_processes 1;
pid "/mnt/c/Users/gonza/OneDrive/Desktop/Sistemas Distribuidos/clase_22/nginx/nginx-wsl.pid";
error_log /tmp/clase22-nginx-error.log;

events {
    worker_connections 1024;
}

http {
    default_type application/octet-stream;
    access_log off;

    server {
        listen 8090;
        server_name localhost;

        location / {
            root "/mnt/c/Users/gonza/OneDrive/Desktop/Sistemas Distribuidos/clase_22/web";
            index index.html;
        }

        location = /dispatcher {
            proxy_pass http://127.0.0.1:8080;
            proxy_http_version 1.1;
            proxy_read_timeout 300s;
            proxy_send_timeout 300s;
        }
    }
}
```

Código fuente actual reproducido como imagen. Las líneas largas se ajustan visualmente; el archivo original no se modifica.

Ejercicio 3 | Configuración validada

La terminal muestra la configuración de NGINX en WSL y su validación: **syntax is ok** y **test is successful**. Se observan el puerto 8090, el directorio de la interfaz y el destino del proxy.

```
wsl: Failed to translate 'D:\cursor\resources\app\bin'
AlbertoGM@BlackRyuPC:/mnt/c/Users/gonza/OneDrive/Desktop/Sistemas Distribuidos$ clase_22
clase_22: command not found
AlbertoGM@BlackRyuPC:/mnt/c/Users/gonza/OneDrive/Desktop/Sistemas Distribuidos$ cd clase_22
AlbertoGM@BlackRyuPC:/mnt/c/Users/gonza/OneDrive/Desktop/Sistemas Distribuidos/clase_22$ nginx -p "$PWD/" -c nginx/nginx-wsl.conf
AlbertoGM@BlackRyuPC:/mnt/c/Users/gonza/OneDrive/Desktop/Sistemas Distribuidos/clase_22$ cat nginx/nginx-wsl.conf
nginx -t -p "$PWD/" -c nginx/nginx-wsl.conf
worker_processes 1;
pid "/mnt/c/Users/gonza/OneDrive/Desktop/Sistemas Distribuidos/clase_22/nginx/nginx-wsl.pid";
error_log /tmp/clase22-nginx-error.log;

events {
    worker_connections 1024;
}

http {
    default_type application/octet-stream;
    access_log off;

    server {
        listen 8090;
        server_name localhost;

        location / {
            root "/mnt/c/Users/gonza/OneDrive/Desktop/Sistemas Distribuidos/clase_22/web";
            index index.html;
        }

        location = /dispatcher {
            proxy_pass http://127.0.0.1:8080;
            proxy_http_version 1.1;
            proxy_read_timeout 300s;
            proxy_send_timeout 300s;
        }
    }
}
nginx: the configuration file /mnt/c/Users/gonza/OneDrive/Desktop/Sistemas Distribuidos/clase_22/nginx/nginx-wsl.conf syntax is ok
nginx: configuration file /mnt/c/Users/gonza/OneDrive/Desktop/Sistemas Distribuidos/clase_22/nginx/nginx-wsl.conf test is successful
AlbertoGM@BlackRyuPC:/mnt/c/Users/gonza/OneDrive/Desktop/Sistemas Distribuidos/clase_22$
```

Ejercicio 3 | Workers y dispatcher

Los workers reciben POST /task en los puertos 8081 y 8082. En la misma prueba, el despachador encuentra **abaco** con 2 de 26 tareas y devuelve **0.846901 s**. Esta medición corresponde a esta captura; la consulta final de la página siguiente es una ejecución diferente.

```
wsl: Failed to translate 'D:\cursor\resources\app\bin'
AlbertoGM@BlackRyuPC:/mnt/c/Users/gonza/OneDrive/Desktop/Sistemas Distribuidos$ cd clase_22
AlbertoGM@BlackRyuPC:/mnt/c/Users/gonza/OneDrive/Desktop/Sistemas Distribuidos/clase_22$ java -cp classes ServidorHTTP 8081
Servidor iniciado: http://127.0.0.1:8081
Endpoints: GET /status, POST /task, GET /ipn
Detener con Ctrl+C.
/127.0.0.1:46488 POST /task
```

```
wsl: Failed to translate 'D:\cursor\resources\app\bin'
AlbertoGM@BlackRyuPC:/mnt/c/Users/gonza/OneDrive/Desktop/Sistemas Distribuidos$ cd clase_22
AlbertoGM@BlackRyuPC:/mnt/c/Users/gonza/OneDrive/Desktop/Sistemas Distribuidos/clase_22$ java -cp classes ServidorHTTP 8082
Servidor iniciado: http://127.0.0.1:8082
Endpoints: GET /status, POST /task, GET /ipn
Detener con Ctrl+C.
/127.0.0.1:52052 POST /task
```

```
wsl: Failed to translate 'D:\cursor\resources\app\bin'
AlbertoGM@BlackRyuPC:/mnt/c/Users/gonza/OneDrive/Desktop/Sistemas Distribuidos$ cd clase_22
AlbertoGM@BlackRyuPC:/mnt/c/Users/gonza/OneDrive/Desktop/Sistemas Distribuidos/clase_22$ java -cp classes WebServer 8080
Dispatcher en http://localhost:8080/dispatcher
Workers: [http://localhost:8081/task, http://localhost:8082/task]
Nueva consulta: 6e44571c522467adc2caf515de7e37f1cf7f17ff4d19d3c3608cf2ab888acb79
Tarea 1 (6e44571c522467adc2caf515de7e37f1cf7f17ff4d19d3c3608cf2ab888acb79,a) -> http://localhost:8081/task
Tarea 2 (6e44571c522467adc2caf515de7e37f1cf7f17ff4d19d3c3608cf2ab888acb79,b) -> http://localhost:8082/task
Tarea 1 terminada: abaco
Tarea 2 terminada: NULL
Tarea 1 (6e44571c522467adc2caf515de7e37f1cf7f17ff4d19d3c3608cf2ab888acb79,a) [http://localhost:8081/task]: abaco
Tarea 2 (6e44571c522467adc2caf515de7e37f1cf7f17ff4d19d3c3608cf2ab888acb79,b) [http://localhost:8082/task]: NULL
Contrasena: abaco
Tareas realizadas: 2 / 26
Tiempo total: 0.846901 segundos
```

Ejercicio 3 | Consulta final

La captura conjunta de navegador y terminal se presenta a continuación en formato horizontal para conservar su legibilidad. Ambos muestran el mismo hash, la contraseña **abaco**, **2 / 26** tareas y el tiempo **0.795952 segundos**.

Dato observado	Resultado
Entrada al sistema	http://localhost:8090, servido por NGINX.
Worker :8081	Tarea con primera letra a: abaco.
Worker :8082	Tarea con primera letra b: NULL.
Resultado final	Contraseña: abaco.
Tareas realizadas	2 / 26; los nuevos envíos se detuvieron.
Tiempo total	0.795952 segundos, igual en navegador y dispatcher.

Interpretación

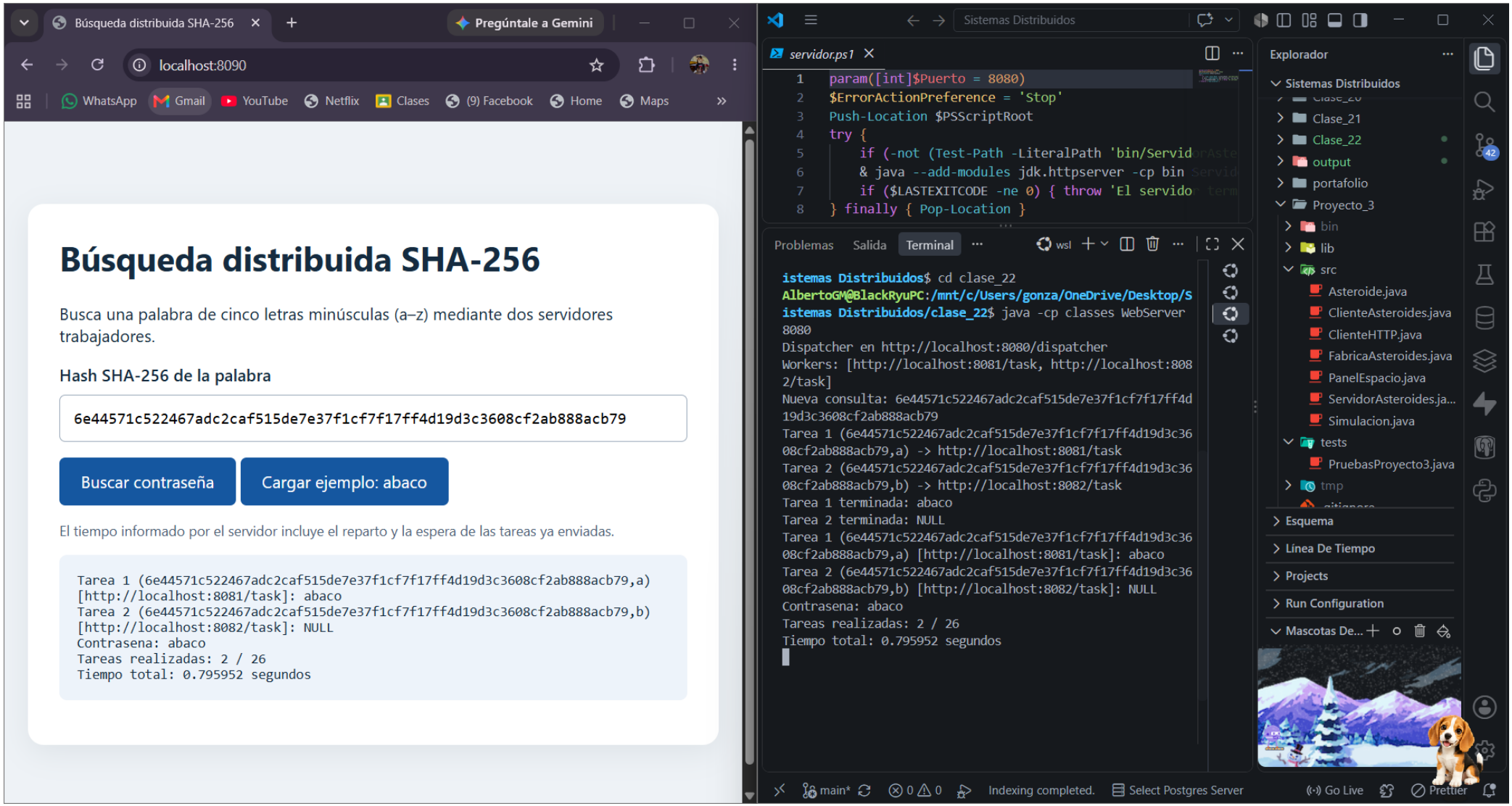
La coincidencia de los datos en navegador y terminal vincula la solicitud de la interfaz con el procesamiento del coordinador. El resultado NULL de la tarea b es esperado: la palabra buscada comienza con a. Las 24 tareas restantes no se enviaron después de encontrar la solución.

Papel de NGINX

El navegador accede al puerto 8090. NGINX entrega la página y reenvía la consulta a /dispatcher. La planificación, la división del problema y la decisión de parada se implementan en Java; NGINX no reemplaza al despachador.

Ejercicio 3 | Navegador y dispatcher en una misma captura

Consulta de abaco: 2 / 26 tareas y 0.795952 segundos en ambos resultados.



Apoyo | Procesamiento del worker

Método POST /task del worker reutilizado de la práctica anterior. Recibe hash,letra, limita el tamaño del cuerpo y llama al motor con una sola primera letra. Devuelve la palabra o NULL y añade el tiempo en X-debug-info.

source/ServidorHTTP.java

```
68      /** POST /task: cuerpo sin claves ni formulario, en formato hashObjetivo,letra. */
69      private static void buscarHash(HttpExchange e) throws IOException {
70          long inicio = System.nanoTime();
71          int estado = 200;
72          String respuesta;
73          try {
74              byte[] cuerpo = e.getRequestBody().readNBytes(4097);
75              if (cuerpo.length > 4096) {
76                  estado = 413;
77                  respuesta = "Cuerpo demasiado grande\n";
78              } else {
79                  String[] datos = new String(cuerpo,
    StandardCharsets.UTF_8).trim().split(",", -1);
80                  if (datos.length != 2 || !datos[1].trim().matches("[a-z]")) {
81                      throw new IllegalArgumentException("Formato esperado:
    hashObjetivo,letra (a-z)");
82                  }
83                  char letra = datos[1].trim().charAt(0);
84                  MotorHash.Resultado resultado = MotorHash.buscar(datos[0].trim(),
    letra, letra);
85                  respuesta = resultado.palabra == null ? "NULL" : resultado.palabra;
86              }
87          } catch (IllegalArgumentException ex) {
88              estado = 400;
89              respuesta = ex.getMessage() + "\n";
90          } catch (Exception ex) {
91              estado = 500;
92              respuesta = "Error interno\n";
93              ex.printStackTrace();
94          }
95          long tiempoNs = System.nanoTime() - inicio;
96          e.getResponseHeaders().set("X-debug-info", String.format(Locale.ROOT,
97                  "seconds=%.6f; milliseconds=%.3f", tiempoNs / 1_000_000_000.0,
    tiempoNs / 1_000_000.0));
98          texto(e, estado, respuesta);
99      }
```

Código fuente actual reproducido como imagen. Las líneas largas se ajustan visualmente; el archivo original no se modifica.

Apoyo | Motor de búsqueda

Fragmento de MotorHash utilizado realmente por ServidorHTTP. Recorre las cuatro posiciones restantes con ciclos anidados y compara los 32 bytes del hash SHA-256 de cada candidato con el objetivo.

source/ServidorHTTP.java

```
147     public static class Resultado {
148         public final String palabra;
149         public final long intentos;
150         Resultado(String palabra, long intentos) { this.palabra = palabra;
    this.intentos = intentos; }
151     }
152     public static Resultado buscar(String hash, char desde, char hasta) throws
    Exception {
153         if (desde < 'a' || hasta > 'z' || desde > hasta)
154             throw new IllegalArgumentException("Rango invalido");
155         byte[] objetivo = decodificar(hash);
156         MessageDigest md = MessageDigest.getInstance("SHA-256");
157         byte[] candidato = new byte[5];
158         long intentos = 0;
159         for (char a = desde; a <= hasta; a++) {
160             candidato[0] = (byte) a;
161             for (char b = 'a'; b <= 'z'; b++) {
162                 candidato[1] = (byte) b;
163                 for (char c = 'a'; c <= 'z'; c++) {
164                     candidato[2] = (byte) c;
165                     for (char d = 'a'; d <= 'z'; d++) {
166                         candidato[3] = (byte) d;
167                         for (char e = 'a'; e <= 'z'; e++) {
168                             candidato[4] = (byte) e;
169                             intentos++;
170                             if (Arrays.equals(md.digest(candidato), objetivo))
171                                 return new Resultado(new String(candidato,
    StandardCharsets.UTF_8), intentos);
172                         }
173                     }
174                 }
175             }
176         }
177         return new Resultado(null, intentos);
178     }
179     }
180 }
```

Código fuente actual reproducido como imagen. Las líneas largas se ajustan visualmente; el archivo original no se modifica.

Conclusiones y reproducción

El sistema cumple la distribución de 26 subtareas entre dos workers y la parada de nuevos envíos cuando se encuentra una palabra válida. La versión final conserva las respuestas de las peticiones que ya estaban en curso y las presenta en el orden de las tareas.

El endpoint /dispatcher permite solicitar el procesamiento por HTTP. La interfaz y el proxy inverso permiten completar el flujo desde el navegador, con resultado y tiempo coincidentes en la terminal del coordinador. La configuración de NGINX fue validada y las capturas muestran su uso en WSL.

Los tiempos de 0.846901 s y 0.795952 s corresponden a consultas distintas de abaco. Las mediciones varían con la carga y los recursos disponibles; ejecutar dos workers en una misma computadora no garantiza reducir el tiempo a la mitad.

Compilación y ejecución

Desde Clase_22: `javac -encoding UTF-8 -d classes source/*.java`

Terminal 1: `java -cp classes ServidorHTTP 8081`

Terminal 2: `java -cp classes ServidorHTTP 8082`

Ejercicio 1: `java -cp classes Application`

Terminal 3: `java -cp classes WebServer 8080`

NGINX: `nginx -p "$PWD/" -c nginx/nginx-wsl.conf`

Interfaz: `http://localhost:8090`

Archivos del proyecto

Los archivos de código están en source/, la interfaz en web/index.html y la configuración de WSL en nginx/nginx-wsl.conf. Los programas Java se compilan en classes/. Las imágenes incorporadas al reporte se conservan en evidencias/entrega/.

Referencia

M. en C. Ukranio Coronilla, Procesamiento Distribuido, Parte 2. Enunciado proporcionado para la práctica. Base de los workers: práctica de Clase_21.